

Module 1 Report: Core Renderers

Student Number: S2510156

October 8, 2026

1 Introduction and AI Usage Declaration

This report documents the implementation of the Whitted-Style Ray Tracer (WSRT) and Distributed Ray Tracer (DRT) for Module 1.

AI Usage Declaration: I used an AI coding assistant (Claude Sonnet 4.6) to help generate initial boilerplate code for the PBRT parser and the basic ray-sphere intersection logic. I also used it to debug the refraction logic. The core integrator loops, the BRDF implementation, and the BVH (for future modules) were written and modified manually. All rendered images and scene files were created by me. The text of this report was written entirely by me without AI assistance.

2 WSRT (Whitted-Style Ray Tracer)

2.1 Specular Reflection

Design Decisions: I implemented recursive reflection using the formula $\mathbf{d}_r = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$. The recursion depth is limited to a configurable maximum (MD=8 for simple, MD=12 for creative) to prevent infinite loops. The throughput is multiplied by the material's reflectance at each bounce. **Difficulties Encountered:** Initially, I passed the incident ray direction directly to the reflection function, which caused incorrect shading. I fixed this by ensuring the dot product uses the outward-facing normal. I also fixed a bug where depth termination returned the throughput as radiance, causing false brightness. **Example Render:**

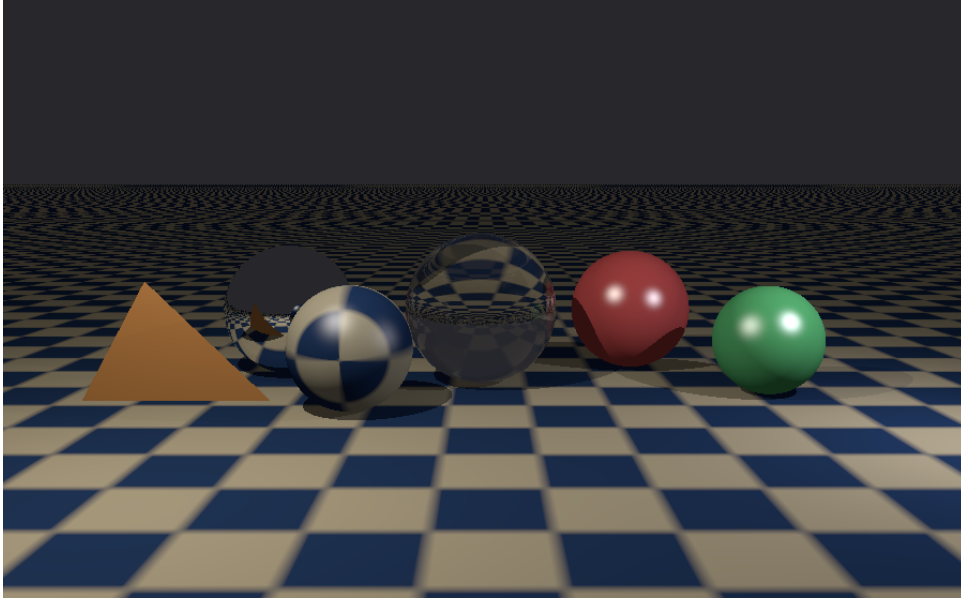


Figure 1: WSRT_simple. The arrow points to the mirror sphere showing reflections of the floor and other objects.

Observations: The mirror sphere correctly reflects the checkerboard floor.

2.2 Specular Refraction (Fresnel and TIR)

Design Decisions: I used Snell's Law ($\eta_i \sin \theta_i = \eta_t \sin \theta_t$) and the Fresnel equations to compute the reflection/refraction ratio. Total Internal Reflection (TIR) is handled when $\sin^2 \theta_t \geq 1$. The medium is determined by checking $\text{dot}(\mathbf{ng}, \mathbf{ray.d}) < 0$. The transmission weight is scaled by $(\eta_i/\eta_t)^2$. **Difficulties Encountered:** I had bugs with the sign of the refraction direction and the handling of the normal when exiting the medium. I also had to fix the DRT integrator which was discarding the weaker Fresnel branch. **Example Render:**

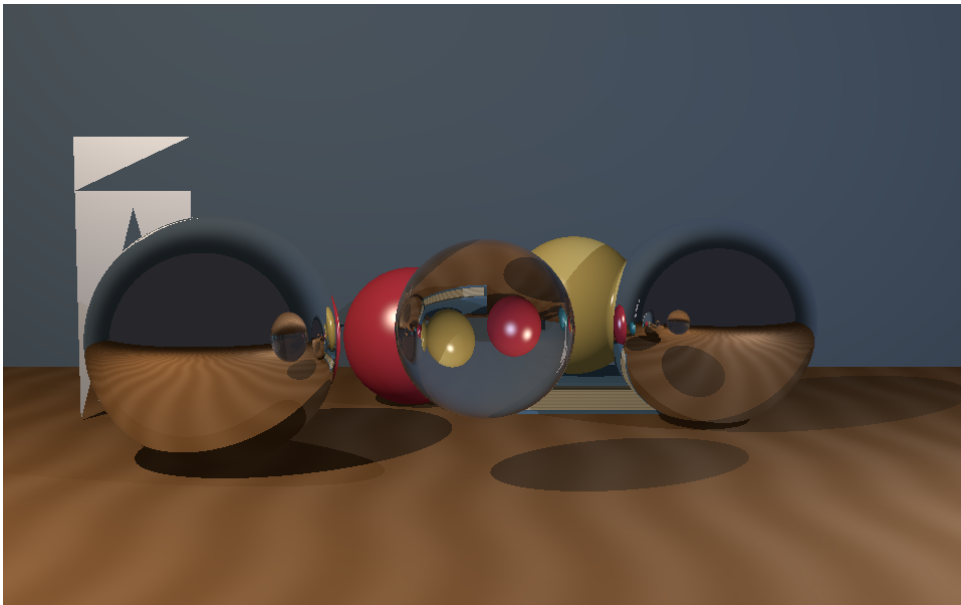


Figure 2: WSRT_creative. The arrow indicates the glass slab where refraction and internal reflection are visible.

Observations: The glass slab ($\eta=1.60$) shows distorted background patterns due to refraction.

2.3 Blinn-Phong Shading

Design Decisions: Ambient is $\text{baseColor} \times \text{ambientScale}$. Diffuse uses $\text{baseColor} / \pi \times \max(0, \mathbf{n} \cdot \mathbf{wi}) \times I / r^2$. Specular uses the half-vector $\mathbf{h} = \text{normalize}(\mathbf{wo} + \mathbf{wi})$ with $\max(0, \mathbf{n} \cdot \mathbf{h})^{\text{shininess}}$. **Example Render:** In Figure 1, the red sphere ($\text{shininess}=120$) and green sphere ($\text{shininess}=60$) show distinct specular highlights.

2.4 Point Light Sources and Shadows

Design Decisions: I implemented point lights with intensity I and distance attenuation I/r^2 . Shadow rays are cast from the intersection point towards the light, with a maximum distance of $\text{dist} - 2\epsilon$ to avoid self-shadowing. The shadow ray origin is offset by $\mathbf{p} + \mathbf{oriented_ng} \times \epsilon$, where $\epsilon = 10^{-4} \times \max(1, \text{dist})$. **Example Render:** The hard shadows cast by the triangle and spheres on the floor in Figure 1 demonstrate this feature.

2.5 Textures and UV Mapping

Design Decisions: Spheres use $u = \phi/(2\pi)$ and $v = \arccos(n.z)/\pi$. Planes use local UV. Textures are loaded from PPM (P3/P6), converted from sRGB to linear, and sampled bilinearly. **Example Render:** The textured sphere in Figure 1 uses a 32x32 checkerboard texture.

2.6 Geometry and Transformations

Design Decisions: I support spheres (quadratic solution), planes ($z=0$ local), and triangles (Möller–Trumbore). Transformations are applied using inverse matrices for rays and inverse-transpose for normals. Ray directions are **not** re-normalized after transformation to keep t consistent between object and world space. **Example Render:** The orange triangle and spheres in Figure 1 demonstrate correct intersection and transformation.

3 DRT (Distributed Ray Tracer)

3.1 Triangle Meshes (OBJ)

Design Decisions: I implemented an OBJ loader supporting v/vt/vn, negative indices, and fan triangulation. Smooth normals are computed as area-weighted cross products when not provided. **Example Render:**

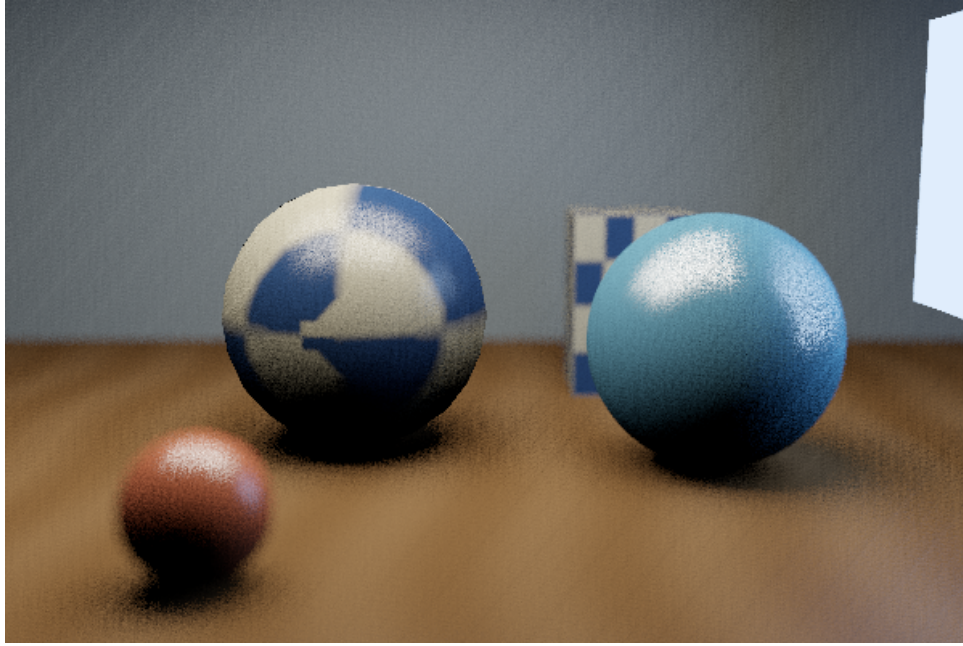


Figure 3: DRT_simple. The arrow points to the OBJ mesh (geodesic sphere) with smooth shading.

3.2 Textures on Meshes

Design Decisions: UV coordinates are interpolated using barycentric weights from the triangle intersection. The base color is computed as $\text{DiffuseColor} \times \text{TextureAt}(\text{uv})$. **Example Render:** In Figure 3, the left OBJ mesh shows a blue/beige checkerboard texture.

3.3 Area Light Sources and Soft Shadows

Design Decisions: I used PCG32 pseudo-random uniform sampling over the area light surface. The solid-angle PDF is $r^2 / (\cos \theta_{\text{light}} \times A)$. The contribution per sample is $f \times \cos \theta_{\text{surface}} \times L_e / \text{pdf} \times \text{beta}$, divided by the number of samples per light. **Difficulties Encountered:** I initially mixed up the row/column coordinates of the rectangle emitter and forgot to transform the sampling normal to world space. **Example Render:**

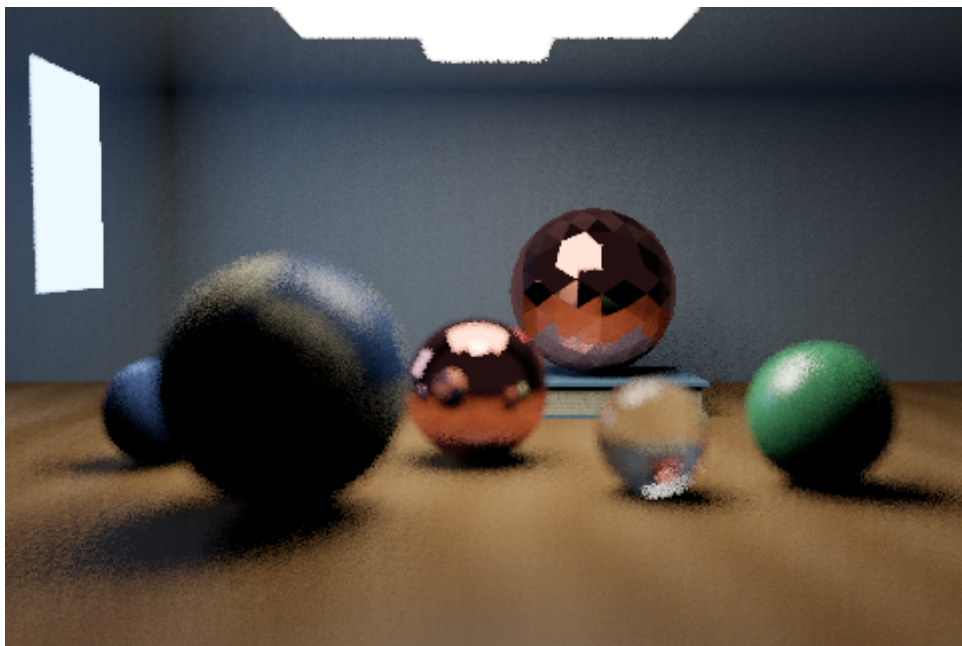


Figure 4: DRT_creative. The arrow indicates the soft shadow (penumbra) cast by the green sphere.

3.4 Phong BRDF and Energy Conservation

Design Decisions: The BRDF is $k_d/\pi + k_s \frac{e+2}{2\pi} \max(0, \text{Reflect}(\mathbf{wi}, \mathbf{n}) \cdot \mathbf{wo})^e$, where $k_s = \text{clamp}(\text{specular}, 0, 1)$ and $k_d = \text{clamp}(\text{texturedDiffuse}, 0, 1) \times (1 - k_s)$. The cosine term is multiplied in the integrator, not in the BRDF. **Example Render:** The glossy highlights on the balls in Figure 4 demonstrate the Phong BRDF.

3.5 Defocus Blur and Depth of Field

Design Decisions: I implemented a thin-lens camera model. Lens sampling uses $r = \text{lensRadius} \times \sqrt{u}$ and $\theta = 2\pi v$. The focal point is $\mathbf{p} + \mathbf{dir} \times t_f$, where $t_f = \text{focusDistance}/(\mathbf{dir} \cdot \text{forward})$. **Difficulties Encountered:** I originally had each ray travel a fixed distance along its own direction, creating a spherical focal surface. I fixed this to intersect a flat focal plane. **Example Render:** In Figure 3 (aperture=0.18, focus=9.0), the foreground red ball and background box are blurred, while the mid-ground is sharp.

3.6 Reinhard Tone Mapping

Design Decisions: I used extended Reinhard tone mapping per RGB channel with white point $W = 8$: $c' = c \times (1 + c/W^2)/(1 + c)$. This is applied after averaging HDR samples and before sRGB encoding. **Example Render:** The white panels in Figure 4 are not blown out, demonstrating the tone mapping.

4 PBRT Scene Compatibility

I support a subset of PBRT syntax including Film, Sampler, Integrator, LookAt, Camera, WorldBegin/End, Material, NamedMaterial, PPM imagemap Texture, point LightSource, diffuse AreaLightSource, sphere, inline trianglemesh, and basic transforms (Translate, Scale, Rotate). **Error Handling:** The parser logs errors for missing textures or unknown shapes.

Unsupported features (e.g., Includes, Object instances) are logged as warnings. A test fixture (`broken.pbrt`) confirms that the renderer handles errors gracefully without crashing.

5 Profiling Results

The following table reports the render time for the four main scenes. Note that the parsing and total timers were not fully implemented at the time of this measurement, so only the render stage is reported. All timings are from a single-threaded run on an Apple M2 Pro.

Scene	Resolution	Samples/Pixel	Max Depth	Primitives	Render Time (s)
WSRT_simple	900x560	1	8	8	0.461692
WSRT_creative	960x600	1	12	47	3.801205
DRT_simple	600x400	32	4	336	66.585001
DRT_creative	480x320	24	4	682	82.613686

Table 1: Render times for Module 1 scenes.

6 Conclusion

In this module, I successfully implemented a Whitted-Style Ray Tracer and a Distributed Ray Tracer from scratch. The WSRT correctly handles specular reflection, refraction, Blinn-Phong shading, shadows, and textures. The DRT extends this with OBJ mesh loading, area light sampling for soft shadows, a Phong BRDF, depth of field, and Reinhard tone mapping.

The main difficulties were fixing the sign conventions for refraction and ensuring consistent t values after transformations. Future work will focus on adding a BVH acceleration structure (Module 3) to handle complex scenes more efficiently.